

Trabalho Prático Grafos

Enzo Casaes Figueiro¹
João Gabriel Santos Andrade Almeida¹
João Victor Filardi Souza Pinto¹
Pedro Henrique Maia Alves¹
Paulo Henrique Fonseca de Assis¹
Rafael Faria Neves Alves Franco¹

¹Pontifícia Universidade Católica de Minas Gerais (PUC-MG)

Resumo. *Este trabalho apresenta o desenvolvimento de uma ferramenta criada para entender como colaboradores interagem dentro de um grande repositório de código aberto no GitHub. A partir da coleta de dados de issues e pull requests, essas interações foram transformadas em grafos que representam desde comentários e revisões até merges, permitindo visualizar como os participantes se conectam e colaboram. Além disso, foi construído um grafo integrado que combina diferentes tipos de relações com pesos distintos, possibilitando análises mais completas. Com base nessa estrutura, foi aplicada métricas de grafos para identificar usuários mais influentes, grupos de trabalho e padrões de colaboração.*

1. Introdução

O crescimento de plataformas de desenvolvimento colaborativo, como o GitHub, tornou essencial compreender como os contribuidores interagem dentro de grandes projetos de código aberto. Issues, pull requests, revisões e comentários formam uma rede complexa de comunicação e cooperação, cujo entendimento pode revelar padrões de colaboração, níveis de engajamento e o papel desempenhado por usuários influentes. No entanto, visualizar e interpretar essas interações de forma estruturada não é trivial, o que motiva a criação de ferramentas capazes de transformar esses dados em representações mais claras e analisáveis, para isso formamos grafos com as devidas informações de cada tópico.

Neste contexto, este trabalho propõe o desenvolvimento de uma ferramenta que utiliza a teoria dos grafos para modelar, analisar e visualizar as interações entre colaboradores de um repositório GitHub. Por meio da coleta automatizada de dados via API pública do GitHub, foram extraídas informações sobre comentários, revisões, fechamentos e merges de issues e pull requests, permitindo representar cada usuário como um nó e cada interação como uma aresta direcionada. O repositório awesome-javascript foi escolhido para o estudo por ser amplamente reconhecido na comunidade, possuir constante atividade e reunir uma grande variedade de ferramentas e bibliotecas populares, características que o tornam ideal para investigar padrões de colaboração em um ambiente real.

A partir dos dados coletados, são construídos grafos parciais para diferentes tipos de interação e, posteriormente, um grafo integrado com pesos definidos conforme a relevância de cada ação. Essa estrutura possibilita identificar colaboradores mais ativos, relações de cooperação e agrupamentos naturais dentro do projeto. O objetivo geral deste trabalho é, portanto, desenvolver uma solução capaz de transformar a dinâmica colaborativa do repositório em uma representação visual e analítica, permitindo investigar a

estrutura da rede, aplicar métricas de centralidade e compreender o funcionamento da comunidade envolvida no projeto.

2. Fundamentação Teórica

2.1. Conceitos Fundamentais de Grafos

Grafos são estruturas matemáticas utilizadas para representar relações entre entidades. Um grafo é composto por um conjunto de vértices (ou nós) e um conjunto de arestas, que conectam pares de vértices. Quando a direção da relação importa, o grafo é classificado como *direcionado*, sendo cada aresta representada como um par ordenado (u, v) . Neste trabalho, utiliza-se um grafo simples, ou seja, sem laços.

Duas características importantes são o *grau de entrada* (in-degree), que corresponde ao número de arestas que chegam ao vértice, e o *grau de saída* (out-degree), que representa as arestas que partem dele. Além disso, grafos contêm pesos nas arestas, permitindo representar a intensidade ou relevância de cada relação, abordagem utilizada na modelagem das interações entre colaboradores do repositório.

Além disso, um grafo pode ser representado de diferentes formas, sendo as mais comuns a *matriz de adjacência* e a *lista de adjacência*. A matriz de adjacência utiliza uma estrutura bidimensional em que cada posição (i, j) indica a presença e, se aplicável, o peso de uma aresta entre os vértices i e j . Essa representação é eficiente para consultas rápidas, porém demanda maior uso de memória, especialmente em grafos esparsos. Já a lista de adjacência organiza, para cada vértice, uma lista contendo todos os seus vizinhos e os pesos associados às arestas, garantindo uma representação mais compacta e adequada a grafos grandes. Essas abordagens permitem visualizar e manipular a estrutura do grafo conforme os requisitos da análise.

2.2. Algoritmos e Métricas de Centralidade

Para compreender a importância estrutural de cada vértice dentro da rede, utilizam-se métricas de centralidade amplamente aplicadas na área de grafos e redes complexas.

2.2.1. Centralidade de Grau (Degree Centrality)

Mede o número de conexões diretas de um vértice. Em grafos direcionados, considera-se separadamente grau de entrada e grau de saída. Essa métrica indica usuários mais ativos ou frequentemente envolvidos nas interações.

2.2.2. Centralidade de Intermediação (Betweenness Centrality)

Avalia quantas vezes um vértice aparece nos caminhos mínimos entre outros pares de nós. Vértices com alta intermediação funcionam como pontes, conectando diferentes regiões da rede.

2.2.3. Centralidade de Proximidade (Closeness Centrality)

Indica o quão próximo um vértice está dos demais, considerando as menores distâncias. Usuários com alto índice de proximidade acessam informações rapidamente dentro da rede.

2.2.4. PageRank / Eigenvector Centrality

Essas métricas avaliam a importância do vértice considerando também a relevância de suas conexões. O PageRank, amplamente conhecido pelo uso no algoritmo de busca do Google, destaca usuários influentes dentro da rede de colaboração.

2.2.5. Densidade da Rede

A densidade reflete o quanto a rede está próxima de sua capacidade máxima de conexões. Redes densas indicam alta colaboração entre os participantes.

2.2.6. Coeficiente de Aglomeração (Clustering Coefficient)

Mede a tendência de formação de grupos coesos. Um alto coeficiente indica que colaboradores que interagem com um mesmo usuário também tendem a interagir entre si.

2.2.7. Modularidade e Detecção de Comunidades

A modularidade permite identificar grupos naturais de usuários que interagem mais entre si do que com o restante da rede. Esses agrupamentos podem representar subequipes ou áreas de atuação dentro do projeto.

2.2.8. Bridging Ties

Representam usuários que conectam diferentes comunidades, sendo essenciais para evitar a fragmentação da rede e facilitar a circulação de conhecimento.

2.3. Ferramentas Utilizadas

2.3.1. GitHub API

A extração dos dados foi realizada utilizando a API pública do GitHub, que permite acessar issues, pull requests, comentários, revisões e merges. Uma classe denominada `GithubClient` foi desenvolvida para gerenciar requisições, interpretar respostas e estruturar os dados obtidos.

2.3.2. Gephi

O Gephi é uma ferramenta amplamente utilizada para visualização e análise de grafos. Ele permite aplicar métricas, manipular layouts e identificar padrões de colaboração. Neste trabalho, será utilizado para visualizar o grafo consolidado gerado pela aplicação.

2.3.3. Estruturas de Grafos Implementadas

A implementação utiliza as seguintes classes definidas no escopo do projeto:

- **AbstractGraph**: define a API comum e métodos auxiliares;
- **AdjacencyMatrixGraph**: representação baseada em matriz de adjacência;
- **AdjacencyListGraph**: representação baseada em listas de adjacência.

Essas classes fornecem funções essenciais para criação, manipulação e exportação dos grafos construídos a partir das interações do repositório.

3. Coleta e Preparação dos Dados

3.1. Análise do Repositório

A coleta de informações teve como base o repositório *awesome-javascript*, escolhido por sua ampla atividade e constante atualização. Esse repositório apresenta grande volume de interações entre usuários, o que o torna adequado para a modelagem de grafos e para a análise da dinâmica colaborativa. A extração dos dados foi realizada por meio da API pública do GitHub, permitindo acessar informações diretamente da plataforma e garantindo precisão e rastreabilidade.

Para viabilizar o processo de mineração, foi desenvolvida a classe `GithubClient`, responsável por enviar requisições à API, interpretar as respostas e organizar os dados em estruturas adequadas. Essa abordagem possibilitou reunir automaticamente informações sobre issues, pull requests e comentários, preservando detalhes como autor, responsáveis pela revisão e demais metadados relevantes.

3.2. Interações Coletadas

As seguintes interações foram extraídas do repositório, por serem representativas das relações colaborativas entre os participantes:

- criação e comentários em *issues*;
- fechamento de issues por usuários distintos do autor;
- criação, comentários e revisões em pull requests;
- aprovações e merges de pull requests.

Cada uma dessas ações representa um tipo específico de colaboração: comentários indicam interação direta, revisões refletem cooperação técnica, e merges indicam validação e integração de contribuições. Dessa forma, as interações coletadas capturam diferentes níveis de participação dos usuários no desenvolvimento do projeto.

3.3. Transformação das Interações em Arestas

As informações obtidas foram organizadas em um grafo direcionado, no qual cada usuário é representado como um nó. As interações entre os colaboradores foram transformadas em arestas conforme a natureza da ação realizada. Por exemplo, quando um usuário comenta em uma issue criada por outro, cria-se uma aresta direcionada do autor do comentário para o criador da issue. Da mesma forma, ações como revisão, aprovação ou merge de pull requests geram conexões direcionadas entre os usuários envolvidos.

Para permitir um modelo mais fiel da colaboração, as arestas receberam pesos distintos de acordo com a relevância da interação:

- comentário em issue ou pull request: peso 2;
- abertura de issue comentada por outro usuário: peso 3;
- revisão ou aprovação de pull request: peso 4;
- merge de pull request: peso 5.

Esses pesos refletem a intensidade da colaboração, destacando ações de maior impacto no processo de desenvolvimento. Com isso, o grafo resultante possibilita analisar não apenas como os usuários se conectam, mas também a força das relações estabelecidas.

4. Modelagem da Solução

4.1. Definição dos Grafos Separados

Para representar adequadamente as interações entre os colaboradores do repositório, foram construídos inicialmente três grafos separados, cada um correspondendo a um tipo específico de relação. Essa abordagem modular permite analisar individualmente os padrões de colaboração antes da integração completa dos dados. Os grafos definidos são:

- **Grafo 1 – Comentários:** representa comentários feitos em issues ou pull requests. Arestas são criadas do autor do comentário para o criador da issue ou PR.
- **Grafo 2 – Fechamento de Issues:** representa a ação de fechar uma issue criada por outro usuário. Uma aresta é criada do usuário que fechou a issue para o autor original.
- **Grafo 3 – Revisões, Aprovações e Merges:** representa ações de maior impacto técnico no fluxo de desenvolvimento. Arestas são criadas do revisor ou responsável pelo merge para o autor da PR.

Cada grafo é direcionado e simples, conforme especificado. Essa separação facilita a interpretação dos dados e a aplicação posterior de pesos específicos durante a integração das relações.

4.2. Definição do Grafo Integrado

Após a construção dos grafos individuais, foi criado um grafo integrado que reúne todas as interações entre os usuários. Nesse grafo consolidado, cada vértice representa um usuário único, e cada aresta representa uma interação resultante de qualquer um dos três grafos separados. As arestas recebem pesos que refletem a relevância da colaboração, definidos da seguinte forma:

- comentário em issue ou pull request: peso 2;
- abertura de issue comentada por outro usuário: peso 3;
- revisão ou aprovação de pull request: peso 4;
- merge de pull request: peso 5.

A soma dos pesos permite quantificar a intensidade da relação entre dois usuários, destacando aqueles que possuem maior envolvimento conjunto no projeto.

4.3. Estrutura de Classes Implementada

A implementação da solução segue uma arquitetura baseada em abstração e herança, conforme solicitado no trabalho. A modelagem inclui as seguintes classes:

- **AbstractGraph:** classe abstrata que define a API comum para todos os tipos de grafos, incluindo métodos essenciais.
- **AdjacencyMatrixGraph:** implementação concreta baseada em matriz de adjacência, ideal para operações que demandam acesso rápido e direto às conexões.
- **AdjacencyListGraph:** implementação baseada em listas de adjacência, mais eficiente em termos de memória para grafos esparsos.

Essa estrutura garante flexibilidade, permitindo que os grafos sejam manipulados de forma consistente em diferentes representações internas.

4.4. Diagramas e Modelagem UML

Para representar graficamente a arquitetura da solução, foram elaborados diagrama UML, incluindo:

- **Diagrama de Classes:** mostrando a relação entre AbstractGraph, AdjacencyMatrixGraph e AdjacencyListGraph, além dos atributos e métodos principais de cada classe.

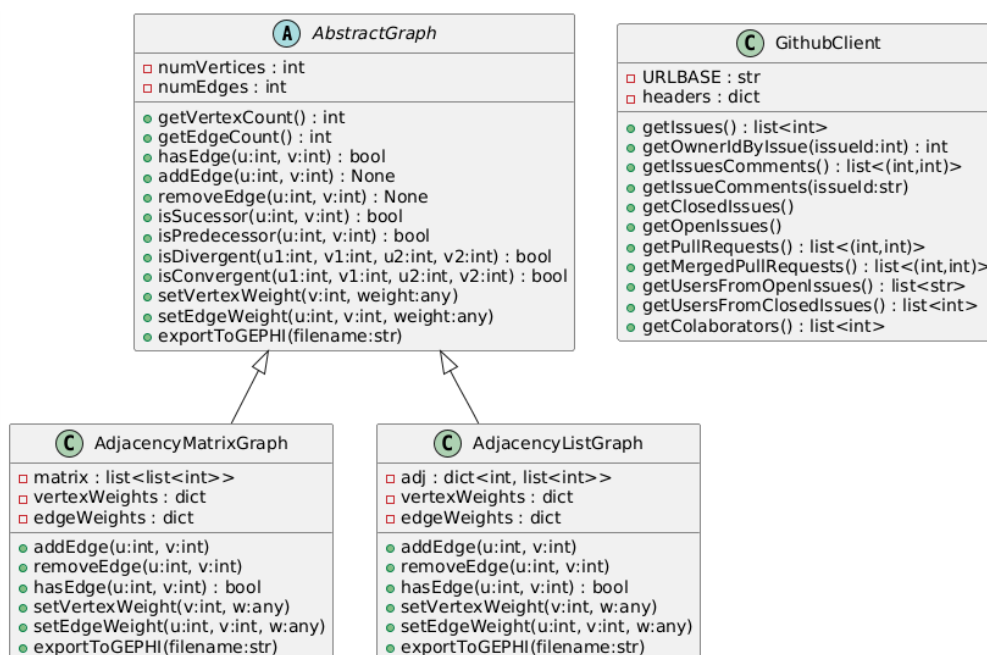


Figure 1. Diagrama de Classes do Sistema.

Esses diagramas auxiliam na compreensão da estrutura do software e das responsabilidades de cada componente, garantindo clareza no processo de desenvolvimento.

5. Análise dos Resultados

A partir do grafo integrado, foram aplicadas diversas métricas de análise de redes para identificar padrões de colaboração, usuários influentes e estruturas internas presentes no repositório. Os resultados podem ser organizados em três grandes grupos: centralidade, estrutura da rede e modularidade/bridging ties.

5.1. Métricas de Centralidade

As métricas de centralidade avaliam a posição e a influência dos usuários dentro do grafo.

5.1.1. Centralidade de Grau (Degree Centrality)

Indica os usuários com maior número de conexões diretas. Valores mais altos sugerem maior atividade ou envolvimento no projeto.

- Maior grau: Usuário 409 (1.0195)
- Valores intermediários: Usuários 100 (0.1488), 61, 75, 187, 191, 196, 198, 201, 206 (0.00976 a 0.0457), indicando conectividade relevante em pontos específicos.
- A maioria dos usuários possui grau muito baixo (0.0 a 0.00488), sugerindo baixa conectividade geral e muitos nós pouco ativos.

5.1.2. Centralidade de Intermediação (Betweenness Centrality)

Identifica usuários que atuam como intermediários entre diferentes partes do grafo.

- Maior betweenness: Usuário 100 (0.0457) e Usuário 409 (0.3010), indicando que eles conectam diferentes subgrupos.
- A grande maioria dos usuários apresenta valor zero, mostrando que poucas pessoas atuam como pontes entre comunidades.

5.1.3. Centralidade de Proximidade (Closeness Centrality)

Avalia a proximidade de cada usuário em relação a todos os outros.

- Todos os valores foram zero, indicando que o grafo possui componentes isolados e muitos nós desconexos.

5.1.4. PageRank

Considera a importância dos vizinhos para avaliar influência.

- Maior PageRank: Usuário 409 (0.3010), seguido por Usuário 100 (0.1488), indicando elevada influência relativa.
- Usuários intermediários: valores entre 0.0024 e 0.0098 (ex.: 61, 75, 187, 191), sugerindo influência moderada.
- A maioria dos usuários possui valores próximos de 0.00488 ou menores, indicando baixa influência relativa.

5.2. Estrutura da Rede e Coesão

Estas métricas refletem características gerais da rede e padrões de interação entre os colaboradores.

5.2.1. Densidade da Rede

A densidade da rede é de 0.00285, indicando que apenas 0.285% das conexões possíveis existem. Apesar de ainda ser uma densidade baixa, ela é superior ao valor anterior, sugerindo uma quantidade um pouco maior de interações entre os usuários. Em repositórios colaborativos amplos, isso continua sendo esperado:

- Nem todos os colaboradores interagem diretamente.
- Muitas contribuições são independentes, especialmente issues e pull requests isolados.
- A existência de alguns usuários mais ativos eleva ligeiramente a conectividade geral.

5.2.2. Coeficiente de Aglomeração (Clustering Coefficient)

Valor = 0.0260, indicando um nível de aglomeração ainda muito baixo, mas agora diferente de zero:

- Há poucos casos em que colaboradores que interagem com um mesmo usuário também interagem entre si.
- Surgem pequenos agrupamentos, porém esparsos e pouco significativos estruturalmente.

O valor permanece consistente com uma rede onde a maior parte das interações é pontual.

5.2.3. Assortatividade

Coeficiente = -0.834, indicando uma desassortatividade ainda mais intensa:

- Usuários muito conectados interagem predominantemente com usuários pouco conectados.
- Os hubs da rede continuam atuando como pontos centrais que recebem interações de muitos colaboradores ocasionais.

Essa forte desassortatividade reforça a estrutura típica de repositórios open source, onde mantenedores experientes revisam ou interagem com contribuições isoladas de usuários menos ativos.

5.3. Métricas de Comunidade

5.3.1. Modularidade

A modularidade observada é -0.0147, demonstrando ausência de comunidades bem estruturadas:

- Atribuição de nós a comunidades pelo algoritmo é artificial.

- Muitos grupos contêm apenas um participante, enquanto outro concentra grande parte dos nós.
- Não há coesão real entre os participantes, refletindo distribuição desorganizada das conexões.

5.3.2. Bridging Ties

Os valores de *bridging ties* são nulos para todos os nós:

- Nenhum participante atua como elo entre comunidades distintas.
- Isso confirma a ausência de fronteiras claras dentro da rede.

6. Resultados Gerais e Conclusão

Com base nas três categorias de métricas analisadas, conclui-se que a rede de colaboração possui as seguintes características:

- **Centralização:** poucos usuários centrais (maintainers) concentram a maior parte das interações, enquanto a maioria é periférica.
- **Esparsidade:** a rede apresenta baixa densidade e mínima formação de clusters.
- **Ausência de comunidades:** modularidade próxima de zero e bridging ties nulos indicam que não existem grupos coesos.
- **Padrão típico de projetos open source grandes:** interações isoladas, revisões centralizadas e colaboradores ocasionais sem interações repetidas.

Esses resultados corroboram a dinâmica observada em repositórios amplamente utilizados, como o *Awesome JavaScript*, mostrando uma rede distribuída, centralizada e com colaboração predominantemente individual.

Essas métricas permitiram extrair insights sobre o comportamento colaborativo do repositório, identificando tanto a distribuição das interações quanto o papel estrutural de cada usuário na rede.

6.1. Interpretação dos Grafos

Os grafos gerados, tanto os parciais quanto o integrado, permitiram visualizar a dinâmica de colaboração entre os participantes do repositório. A análise do grafo integrado mostrou uma rede heterogênea.

A estrutura do grafo revelou:

- A inexistência de comunidades bem definidas;
- Grande interação entre usuários separadamente;

Essa visualização ajudou a destacar padrões de contribuição e a entender como o fluxo de colaboração se espalha pelo projeto.

6.2. Imagens e Visualizações Geradas

As visualizações do grafo integrado foram realizadas utilizando o software Gephi, que permitiu aplicar escalas que facilitaram a visualização do grafo e a colaboração entre os usuários

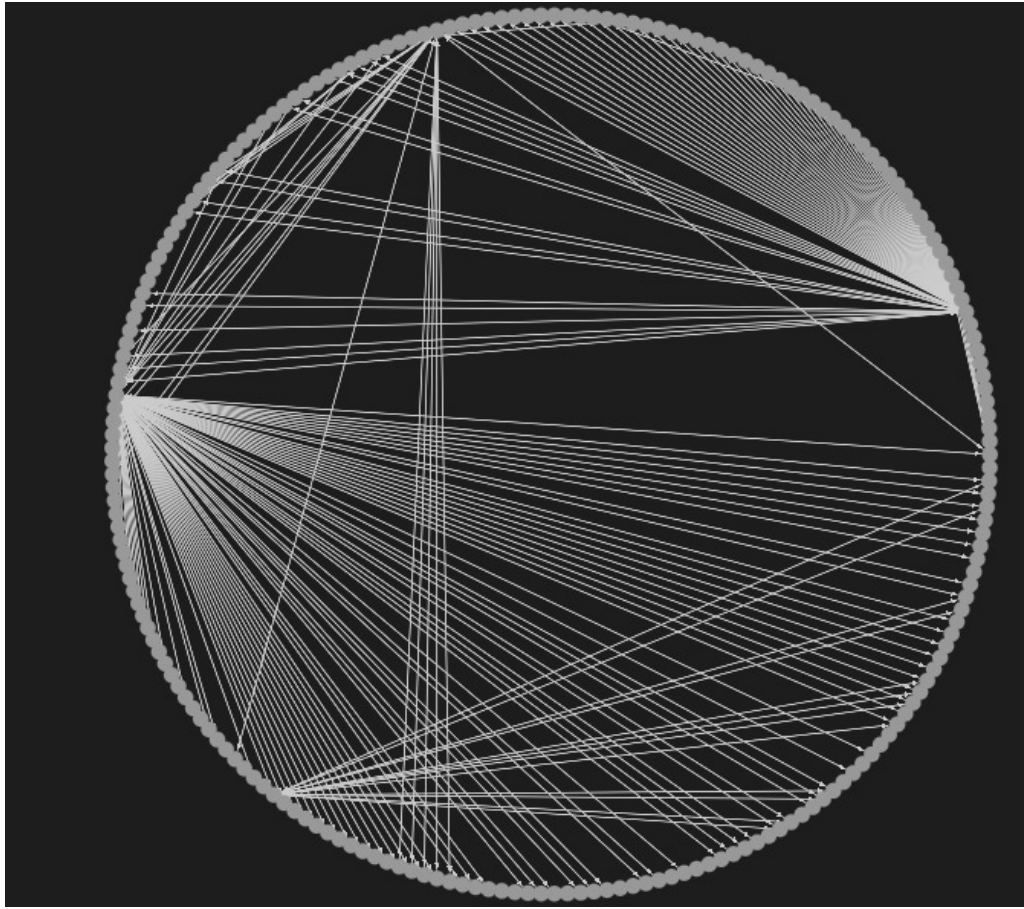


Figure 2. Grafo 1 utilizado na análise do sistema.

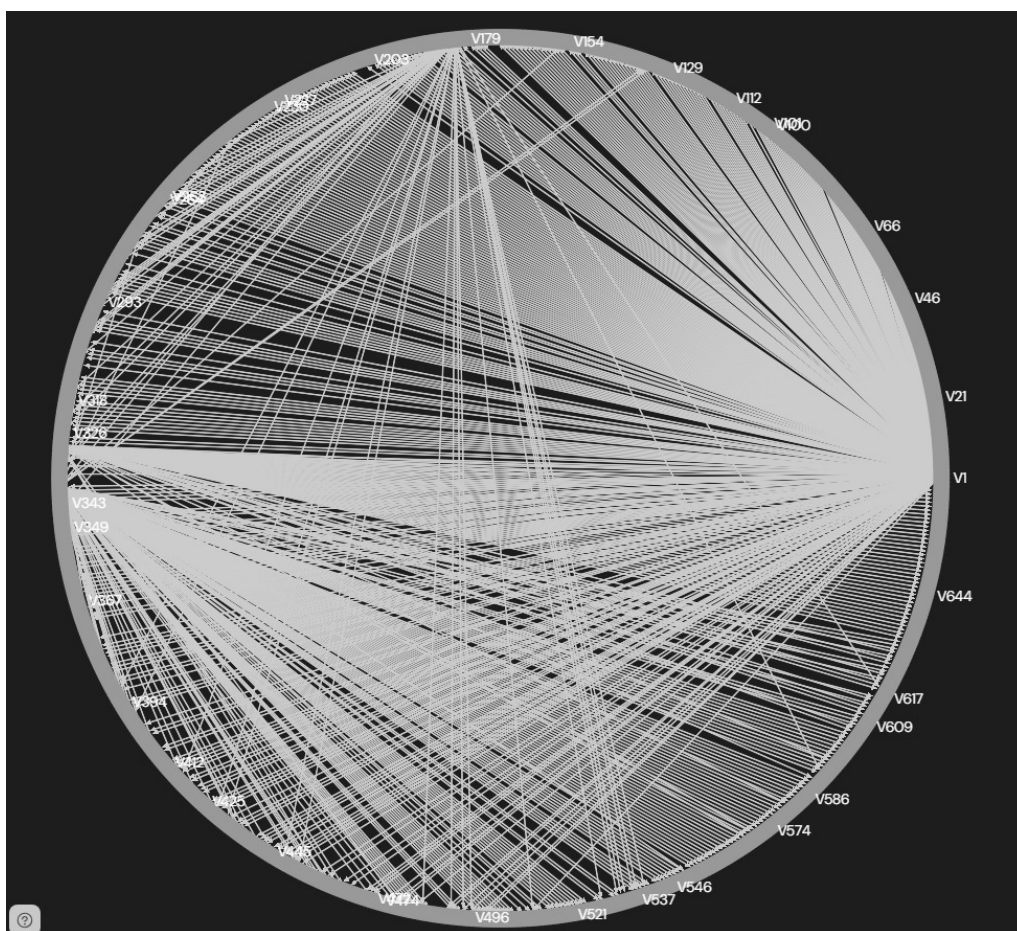


Figure 3. Grafo 2 utilizado na análise do sistema.

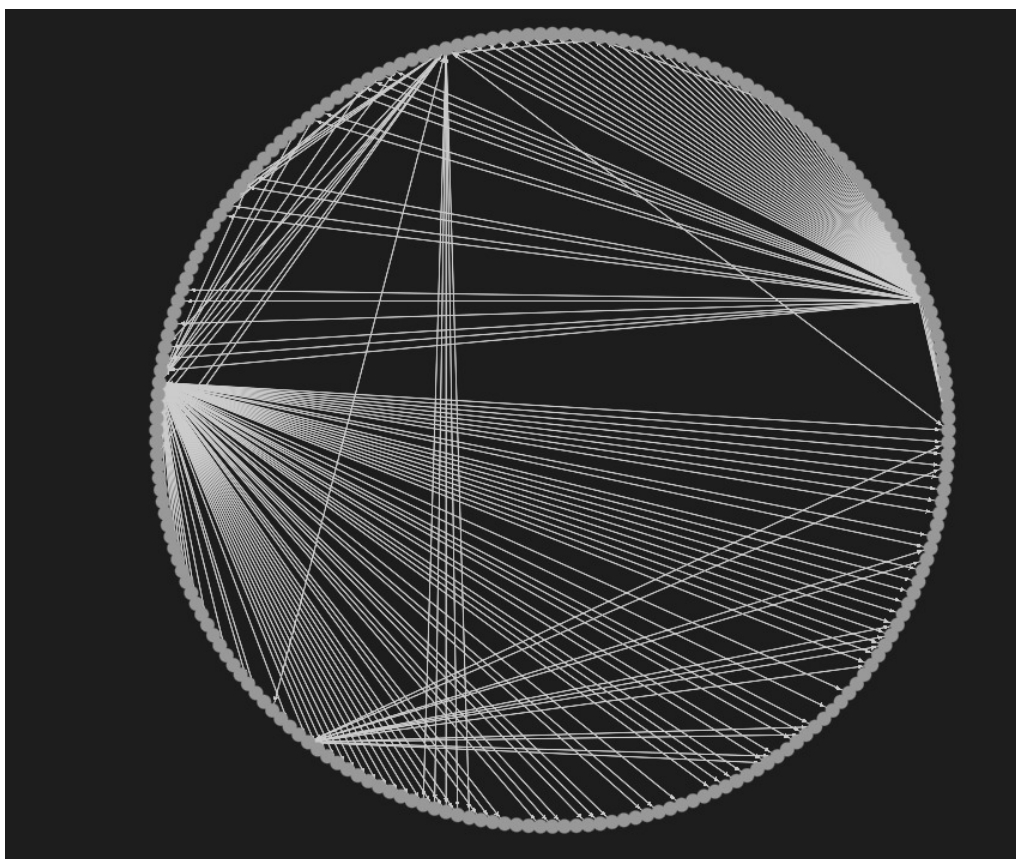


Figure 4. Grafo 3 utilizado na análise do sistema.



Figure 5. Grafo 4 incompleto utilizado na análise do sistema.

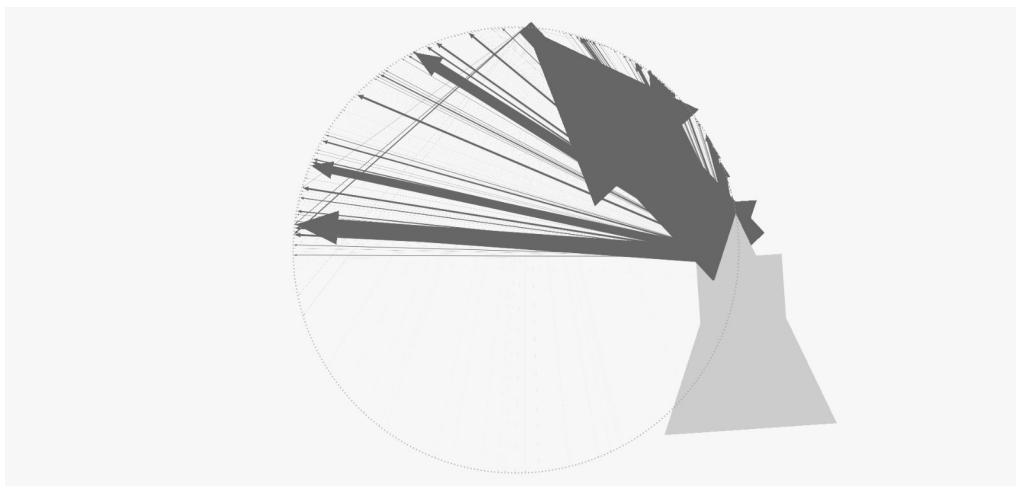


Figure 6. Grafo 4 completo utilizado na análise do sistema.

7. Conclusão

O desenvolvimento deste trabalho permitiu a construção de uma ferramenta capaz de coletar e modelar interações entre colaboradores de um repositório de código aberto utilizando conceitos da teoria dos grafos e redes complexas. A partir da extração automática de dados por meio da API do GitHub, foi possível transformar ações como comentários, revisões, fechamentos e merges em representações estruturadas dentro de grafos direcionados, oferecendo uma visão técnica e visual das relações de colaboração.

Foram implementados grafos separados para diferentes tipos de interação, além de um grafo integrado que consolidou todas as relações com pesos definidos conforme a relevância das ações. Essa abordagem permitiu identificar padrões de comunicação, usuários mais influentes, grupos naturais de trabalho e características estruturais do repositório analisado. Métricas como centralidade, densidade, aglomeração e modularidade foram aplicadas para aprofundar a compreensão da dinâmica colaborativa.

As principais contribuições deste projeto incluem: a construção de uma ferramenta robusta e extensível para análise de interações em repositórios GitHub; a utilização de estruturas de grafos implementadas em código próprio, seguindo princípios de abstração e herança; a capacidade de exportar os resultados para visualização no Gephi. Esses resultados demonstram o potencial da teoria dos grafos como ferramenta de apoio à compreensão de processos colaborativos e reforçam a importância da análise de redes no estudo de projetos abertos.